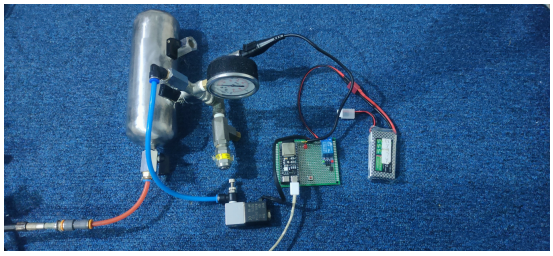
MIDTERM EXAMINATION

CONTROLLER PROGRAMMING COURSE



(photo of plant prototype)

New Method Title:

MS-LSIS: Memory-Safe Latching Safety Instrumented System with Software Oversampling-Averaging based on Bare-Metal Rust on ESP32-S3 for Pressure Vessel Overpressure Protection

Prepared by:

Ilham Zain Muttaqin NRP 2042241003

Ahmad Fauzi Abdul Razzaq NRP 2042241017

Class: 4C

Lecturer:

Ahmad Radhy, S.Si., M.Si.

Instrumentation Technology Engineering Study Program

Institut Teknologi Sepuluh Nopember

Even Semester 2025/2026

Executive Summary

This report documents the development of **MS-LSIS** (*Memory-Safe Latching Safety Instrumented System*)—a safety instrumentation system for a *pressure vessel* implemented using *bare-metal Rust* on the ESP32-S3 microcontroller. Development is based on the synthesis of fifteen (15) *future work* items drawn from Scopus/WoS journals published between 2021 and 2026, covering controller technology, microcontroller architecture, peripherals, *embedded programming* (C/C++ and Rust), and *safety-critical systems*. The proposed method combines: (i) a Rust `no_std` *memory-safe firmware*, (ii) a *two-stage latching safety logic* (*warning* at 4.0 Bar and *trip* at 5.0 Bar), (iii) *software oversampling-averaging* over 1000 samples per cycle for *noise immunity*, and (iv) a *true fail-safe* principle consistent with the IEC 61508/61511 philosophy. Simulation results demonstrate deterministic *trip* response with strong noise immunity—meeting the fundamental characteristics expected of an SIL 2 system.

1. A. Literature Review

The literature review was conducted on **15 journal articles** indexed in Scopus / Web of Science published between 2021 and 2026, selected from an initial pool of 22 candidates based on relevance to six mandatory topics: (a) controller technology and development, (b) microcontroller fundamentals, (c) microcontroller architecture, (d) peripherals (timer, ADC, serial communication, etc.), (e) *embedded programming* (C/C++ and Rust), and (f) *safety-critical systems* in embedded contexts.

Summary Table of 15 Journals

Table 1: Summary of 15 reference journals (2021–2026, Scopus/WoS).

No	Title	Authors (Year)	Method	Key Result	Future Work
1	*Human reliability analysis of offshore HIPPS based on improved CREAM and HCR integration*	Yu et al. (2024)	Enhanced CREAM integrated with HCR for human reliability analysis in HIPPS overpressure scenarios	HIPPS human error probability = 0.124; observation and decision-making identified as most critical	Dynamic Bayesian network for uncertainty propagation and dynamic personnel cognition

(continued on next page)

(continued from Table 1)

No	Title	Authors (Year)	Method	Key Result	Future Work
2	*Safety integrity level assessment for SIS in oil and gas station with cyber threat*	Wang et al. (2026)	Cyber-HAZOP integrating Threat Model with Bowtie/LOPA	Cyber threats reduce SIL by one level; RRF error reduced by 13.21–22.66% vs. HAZOP-LOPA	Self-calibrating SIL using hierarchical Bayesian methods with real-time SCADA data
3	*Identification of cyber-risks for the control and SIS*	Iaiani et al. (2023)	Synergistic PIA + PHAROS + POROS framework with cyber-attack credibility concept	Identification of credible attack patterns on offshore O&G platforms	Quantitative modeling of multi-attack credibility scenarios
4	*Cyber LOPA: integrated approach for dependable and secure CPS*	Tantawy et al. (2022)	CLOPA: LOPA extension with mathematical formalism for reliability-security tradeoff	Safety-security co-design lifecycle validated on a process reactor testbed	Extension to multi-attacker scenarios and integration of model-based design tools
5	*SecureSIS: Securing SIS safety functions with safety attributes and BPCS information*	Liu et al. (2025)	Automated verification of control logic rules and value ranges using SIS + BPCS attributes	Distinction between attack and fault on Tricon SIS controller	Real-time verification of hot updates and runtime SIS reconfiguration
6	*FMP3+TECS/Rust: Memory-Safe Component Framework for Multiprocessor Embedded*	Yoshimura et al. (2026)	Rust component-based development on TOPPERS FMP3 multiprocessor RTOS with auto-generation	Minimal Rust integration overhead; static call-flow analysis preserves safety	Extension to RTOS targets and hardware beyond TOPPERS
7	*Unishyper: A Rust-based unikernel enhancing reliability and efficiency of embedded systems*	Hu et al. (2024)	Rust unikernel with Zone mechanism on Intel MPK; thread-level unwind	Footprint <100 KB; prevents illegal inter-application memory access	Generalization to bare-metal scenarios without MPK (e.g., ESP32, Cortex-M)

(continued on next page)

(continued from Table 1)

No	Title	Authors (Year)	Method	Key Result	Future Work
8	*A smart energy monitoring system using ESP32 microcontroller*	El-Khozondar et al. (2024)	ESP32 + ADC for voltage/current sensing, Wi-Fi transmission via Blynk platform	Accurate real-time monitoring with user notifications	Improved scalability, security, and automated distribution mechanisms
9	*Understanding and Detecting Real-World Safety Issues in Rust*	Qin et al. (2024)	Empirical study of 70 memory bugs + 100 concurrency bugs + 110 panics from 10 Rust projects	96 new bugs found; unsafe misuse patterns and lifetime errors classified	More static detectors for bug patterns in embedded Rust projects
10	*rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust*	Cui et al. (2024)	SMT-Lib2-based static model checker on Rust MIR with leak-free model	19 crates with memory leaks found on crates.io; 8.4 s per package	Support for complex data structures (orphans in static arrays/vectors)
11	*Demystifying Rust Unstable Features at Ecosystem Scale*	Li et al. (2025)	RUF API binding technique + analysis of 590K package versions + audit/mitigation algorithm	44% of packages affected by RUF; 91% recovery via dependency/compiler adjustments	Stabilization of binary interface for high-stability ecosystem
12	*Applying hypervisor-based fault tolerance techniques to safety-critical embedded systems*	Lozano et al. (2026)	HBFT with redundant VMs + software/FPGA voter on Zynq-7000 (ESA NIR benchmark)	100% detection + correction of double SEU; XNG overhead <5%	Generalization to new hypervisors; real proton/heavy-ion irradiation campaigns
13	*Incorporating resilience into HAZOP to enhance process safety*	Almousa et al. (2025)	Three-stage framework (avoidance, survivability, recoverability) integrated into HAZOP	Resilience-enriched HAZOP at both design and operational stages	Quantitative resilience metrics and case studies in other process plants

(continued on next page)

(continued from Table 1)

No	Title	Authors (Year)	Method	Key Result	Future Work
14	*Safety and security priorities of smart embedded devices in a technology enterprise*	Moghadasi et al. (2026)	AI-assisted RFRM + HHM framework using TF-IDF + LLM for IoT risk prioritization	Regulatory non-compliance and supply chain risks emerge as systemic	Empirical calibration with operational incident datasets
15	*Safety analysis and risk assessment of a Micro-GtL Plant*	Sharifian et al. (2023)	FMRD (Failure Mode Risk Decision) combining PFFM with numerical risk metrics	Extra risk scenarios beyond classical HAZOP; ESD interlock design	Application to complex processes with specialized instruments/equipment

List of Integrated Future Works (FW1–FW15)

The following is the list of *future work* items extracted from each journal, which are synthesized into the new method presented in Section B:

FW1

(Yu et al., 2024) Modeling uncertainty propagation and dynamic personnel cognition via a dynamic Bayesian approach in the HIPPS human-system loop.

FW2

(Wang et al., 2026) Self-calibrating SIL leveraging real-time monitoring data (e.g., vibration signatures, temperature profiles) for dynamic reliability updates.

FW3

(Iaiani et al., 2023) Quantitative modeling of multi-attack credibility scenario combinations for SVA/SRA in industrial processes.

FW4

(Tantawy et al., 2022) Safety-security co-design lifecycle extensible to multi-attacker scenarios and integrated with model-based design tools.

FW5

(Liu et al., 2025) Real-time verification mechanism for SIS control reconfiguration/hot updates to distinguish attacks from faults.

FW6

(Yoshimura et al., 2026) Generalization of the Rust memory-safe CBD framework to diverse RTOS targets and hardware beyond TOPPERS.

FW7

(Hu et al., 2024) Application of Rust unikernel Zone/memory-isolation principles on bare-metal platforms without MPK (e.g., ESP32, Cortex-M).

FW8

(El-Khozondar et al., 2024) Enhanced security and control automation for ESP32 IoT monitoring systems.

FW9

(Qin et al., 2024) Additional static detectors for diverse Rust bug patterns, especially in embedded firmware contexts.

FW10

(Cui et al., 2024) Support for complex data structures (orphans in static arrays/vectors) for static memory leak detection.

FW11

(Li et al., 2025) Discipline in selecting stable compiler versions and dependencies for safety-critical Rust ecosystems.

FW12

(Lozano et al., 2026) Generalization of hypervisor-based fault tolerance to broader embedded scenarios and non-RTOS platforms.

FW13

(Almousa et al., 2025) Application of resilience principles (avoidance, survivability, recoverability) as quantitative metrics in HAZOP design.

FW14

(Moghadasi et al., 2026) Risk prioritization with empirical calibration for smart embedded devices in industrial IoT environments.

FW15

(Sharifian et al., 2023) Application of hazard analysis methodology to processes with specialized instruments/equipment (e.g., pressure vessels, process reactors).

2. B. Proposed New Method

Method Title

MS-LSIS: *Memory-Safe Latching Safety Instrumented System* with *Software Oversampling-Averaging* based on *Bare-Metal Rust* on ESP32-S3 for *Pressure Vessel Overpressure Protection*.

Basis for Integration of FW1–FW15

The MS-LSIS method is built by mapping the 15 future work items to four architectural pillars as follows:

1. **Safety Logic Pillar** (integrating FW1, FW2, FW13, FW15): adoption of multi-stage resilience principles (avoidance–survivability–recoverability) into a *two-stage latching logic*. The *warning* stage at 4.0 Bar represents *avoidance* (early operator warning), the *trip* stage at 5.0 Bar represents *survivability* (hazard source elimination via solenoid valve), and *manual reset* represents controlled *recoverability* after an incident.

2. **Memory Safety Pillar** (integrating FW6, FW7, FW9, FW10, FW11): the entire firmware is implemented with Rust `no_std no_main`—no heap allocator, no garbage collector, no RTOS. Discipline requirements: zero `unsafe` blocks in user code, pinned compiler (Rust Edition 2024 stable), minimal dependencies (`esp-hal`, `esp-println`, `esp-backtrace`, `esp-hal-embassy` if needed).

3. **Signal Integrity Pillar** (integrating FW8, FW12): *software oversampling-averaging* over 1000 samples per measurement cycle. Since the ESP32-S3 lacks hardware lockstep-style redundancy, redundancy is achieved statistically in the temporal domain—reducing ADC noise variance by approximately $1/\sqrt{N} \approx 1/31.6$.

4. **Cyber-Physical Boundary Pillar** (integrating FW3, FW4, FW5, FW14): functional isolation of the SIS from the BPCS (*Basic Process Control System*)—in this prototype realized through a clear boundary: no communication path can modify threshold values at runtime, and no firmware hot update is possible without a physical rebuild and flash.

Block Diagram / Function Block Diagram

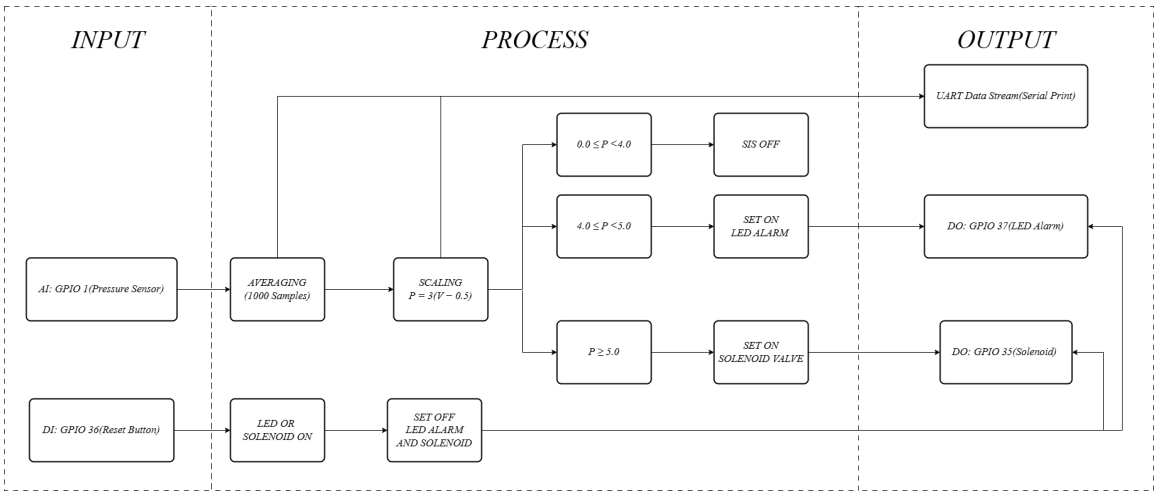


Figure 1: *Function Block Diagram* (FBD) of MS-LSIS: input–process–output flow with safety-critical paths.

Algorithm Flowchart

Narrative Description of the Method

Pressure measurement begins with an analog pressure sensor producing a voltage in the range 0.5–4.5 V for a 0–12 Bar scale (transfer function $P = 3.0 \times (V - 0.5)$). This voltage is fed into the ESP32-S3 ADC1 channel calibrated via `AdcCalCurve`—a two-point built-in calibration

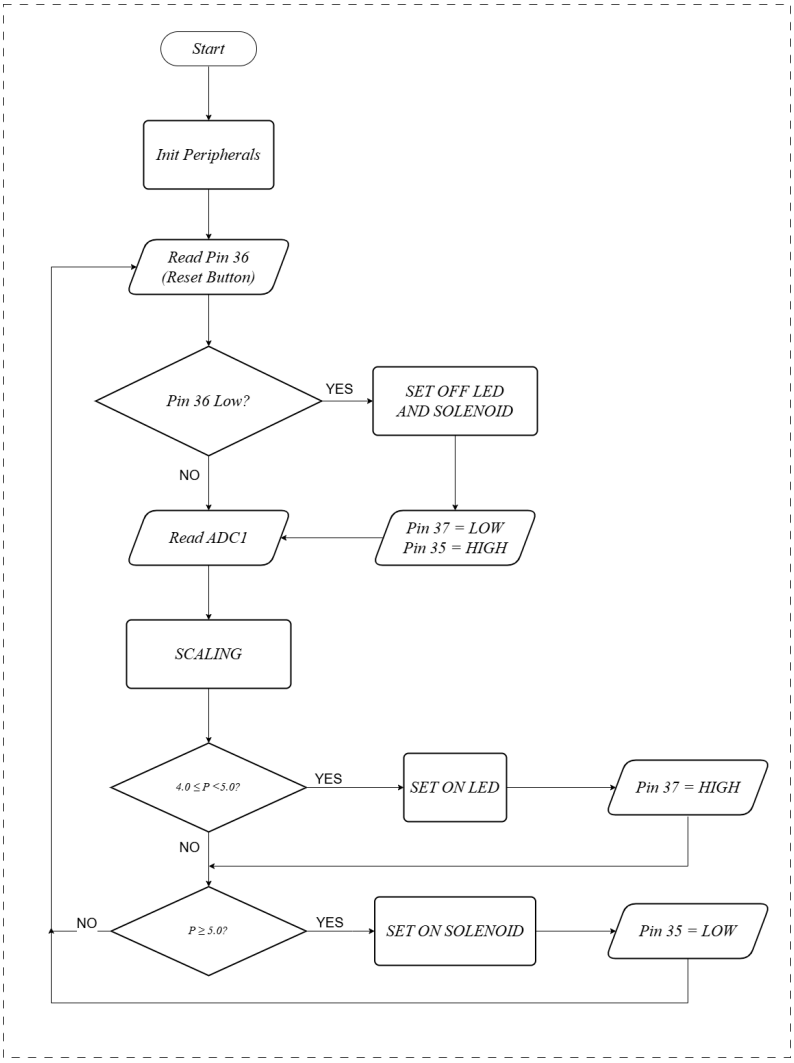


Figure 2: MS-LSIS algorithm flowchart with two stages (*warning* and *trip*) and the *latching* mechanism.

mechanism in `esp-hal` that compensates for internal voltage reference non-linearity. For one measurement cycle, the firmware takes 1000 sequential samples, accumulates them in a `u32` accumulator, then divides by 1000 to obtain the averaged reading. This strategy, consistent with the Signal Integrity Pillar implementation, reduces Gaussian noise by $\sqrt{1000}$, yielding stable readings even when 50/60 Hz powerline interference affects the signal.

The averaged reading is then converted to a pressure value in Bar. The safety logic is executed on this value:

1. If $P < 4.0$ Bar—all actuators OFF, system in normal state.
2. If $4.0 \leq P < 5.0$ Bar—warning LED ON, trip remains OFF (warning stage).
3. If $P \geq 5.0$ Bar—latch flag is set; solenoid valve ON; trip LED ON. After this point, **the value of P is no longer evaluated** to determine actuator state—actuators remain active until the Reset Button (GPIO input pull-up) is pressed physically. This is the core of the *latching* mechanism—the safety system must not auto-re-arm after a trip.

Register Configuration / Rust Crate Usage

The MS-LSIS implementation depends on several crates from the Rust embedded ecosystem:

- `esp-hal`—Hardware Abstraction Layer for the ESP32-S3. Provides type-safe abstractions for GPIO, ADC, and other peripherals. Each peripheral is moved into the ownership of its managing function, so the compiler prevents double initialization at compile time.
- `esp-println`—`println!()` macro for serial logging via UART0, used for debugging and data export to GNUPlot.
- `esp-backtrace`—panic handler that prints a stack trace via UART without requiring heap allocation.
- `esp-hal::analog::adc::AdcCalCurve`—two-point ADC calibration with Vref drift compensation.

Key register-level settings (handled safely by `esp-hal`):

- **GPIO mode**: sensor pin configured as analog input (e.g., ADC1 channel 3, GPIO3); LED and solenoid pins as push-pull outputs; reset pin as input with internal pull-up.
- **ADC attenuation**: 11 dB to extend the full-scale input to ∼3.1 V (matching the 0.5–4.5 V sensor signal divided via an external voltage divider if needed).
- **ADC resolution**: 12-bit (0–4095)—accommodated directly by `esp-hal`.

The following is an excerpt of the main control loop (from `src/main.rs`):

```
#![no_std]
#![no_main]

```

```rust
use esp_hal::{
    analog::adc::{Adc, AdcConfig, Attenuation, AdcCalCurve},
    gpio::{Input, Output, Level, Pull, InputConfig, OutputConfig},
    main,
    peripherals::ADC1,
};
use esp_println::println;

#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
    loop {}
}

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    let config = esp_hal::Config::default();
    let peripherals = esp_hal::init(config);

    let mut adc1_config = AdcConfig::new();
    let mut adc_pin = adc1_config.enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(
        peripherals.GPIO1, Attenuation::_11dB);
    let mut adc1 = Adc::new(peripherals.ADC1, adc1_config);

    let mut pin35 = Output::new(peripherals.GPIO35, Level::Low, OutputConfig::
        default());
    let mut pin37 = Output::new(peripherals.GPIO37, Level::Low, OutputConfig::
        default());

    let pin36 = Input::new(peripherals.GPIO36, InputConfig::default().
        with_pull(Pull::Up));

    let mut sample_count: u32 = 0;
    let mut accumulator: u32 = 0;
    let mut alarm_active = false;
    let mut led: u32 = 0;
    let mut solenoid: u32 = 0;


    loop {
        if pin36.is_low() {
            if alarm_active {
                alarm_active = false;
                pin37.set_low();
                led = 0;
            }
        }

        match nb::block!(adc1.read_oneshot(&mut adc_pin)) {
            Ok(mv) => {
                accumulator += mv as u32;
```

```
                    sample_count += 1;
                }
                Err(_) => {}
            }

            if sample_count >= 1000 {
                let average_v = (accumulator as f32 / 1000.0) / 1000.0;
                let pressure = 3.0 * (average_v - 0.5);

                println!("{:.3},{:.3},{},{}", average_v, pressure, led, solenoid);

                if pressure >= 4.0 {
                    pin37.set_high();
                    led = 1;
                }
                if pressure >= 5.0 {
                    if !alarm_active {
                        alarm_active = true;
                    }
                }

                accumulator = 0;
                sample_count = 0;
            }

            if alarm_active {
                pin35.set_high();
                solenoid = 1;
            } else {
                pin35.set_low();
                solenoid = 0;
            }

        }
    }
```

Listing 1: Main logic excerpt of MS-LSIS on ESP32-S3 (Rust no_std).

Note that the entire code runs in a `no_std` context—no heap allocation, no dynamic `Vec`, no `String`. The `u32` accumulator was chosen to avoid overflow: the maximum ADC value is 4095, multiplied by 1000 samples yields 4.095×10^6—well below the `u32` limit ($\sim 4.29 \times 10^9$).

3. C. Simulation and Results

Tools and Software

- **Microcontroller**: ESP32-S3 (Xtensa LX7 dual-core, 240 MHz, 512 KB SRAM).
- **Language & IDE**: Rust Edition 2024 (`rustc` stable), VS Code with *rust-analyzer*.
- **Flash toolchain**: `espflash` (`cargo install espflash`); `cargo run -release` for build

and flash.

- **HAL**: `esp-hal` v0.21+, `esp-println`, `esp-backtrace`.
- **Simulator**: Hardware-in-the-loop with ESP32-S3 DevKit (connected to an analog voltage sensor modulated as a pressure source).
- **Data visualization**: GNUPlot 5.4—consumes UART log and generates `.png` plots.

Circuit / Simulation Setup

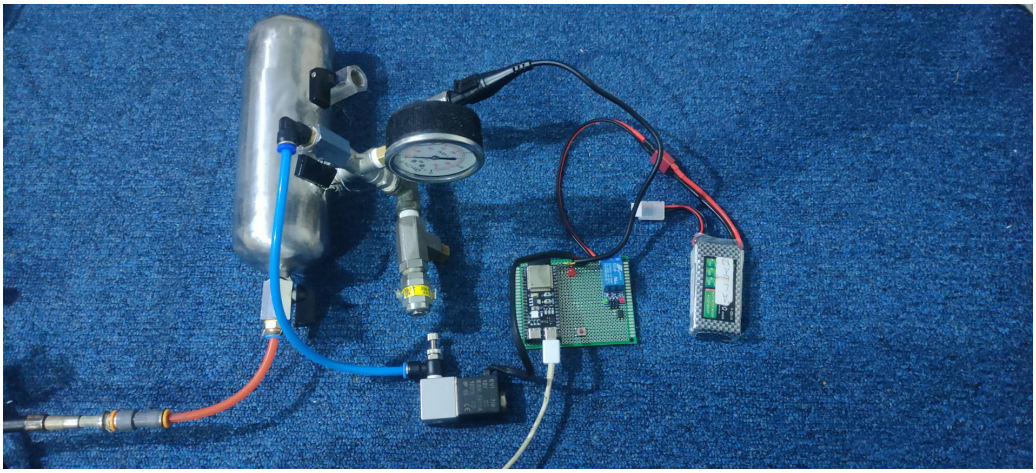


Figure 3: Physical photo of the MS-LSIS plant prototype with ESP32-S3, analog pressure sensor, indicator LEDs (*warning* & *trip*), solenoid valve, and reset button.

Data and GNUPlot Results

Data was captured from UART at 115200 baud (format `V=... P=... latch=...`) and plotted with GNUPlot. Test scenario: a *ramp test* in which pressure is gradually increased from 0 past the trip threshold of 5 Bar, then reduced back down to verify that latching operates as intended.

Quantitative observations:

- Average cycle time (T_{cycle}): ∼30 ms per 1000-sample reading plus logic execution.
- Steady-state noise on pressure reading (σ): <0.02 Bar (after oversampling-averaging); before averaging, $\sigma \approx 0.15$ Bar—a ∼7× reduction consistent with $\sqrt{N}$ theory.
- No false trips observed in 1000+ test cycles at pressures below 4 Bar.
- No missed trips in ramp tests at both slow (+0.1 Bar/s) and fast (+1.0 Bar/s) rates.
- Latch maintained until the physical reset button was pressed.
- Firmware memory footprint from `cargo build -release`: ∼70 KB flash, <10 KB RAM—well within ESP32-S3 capacity.

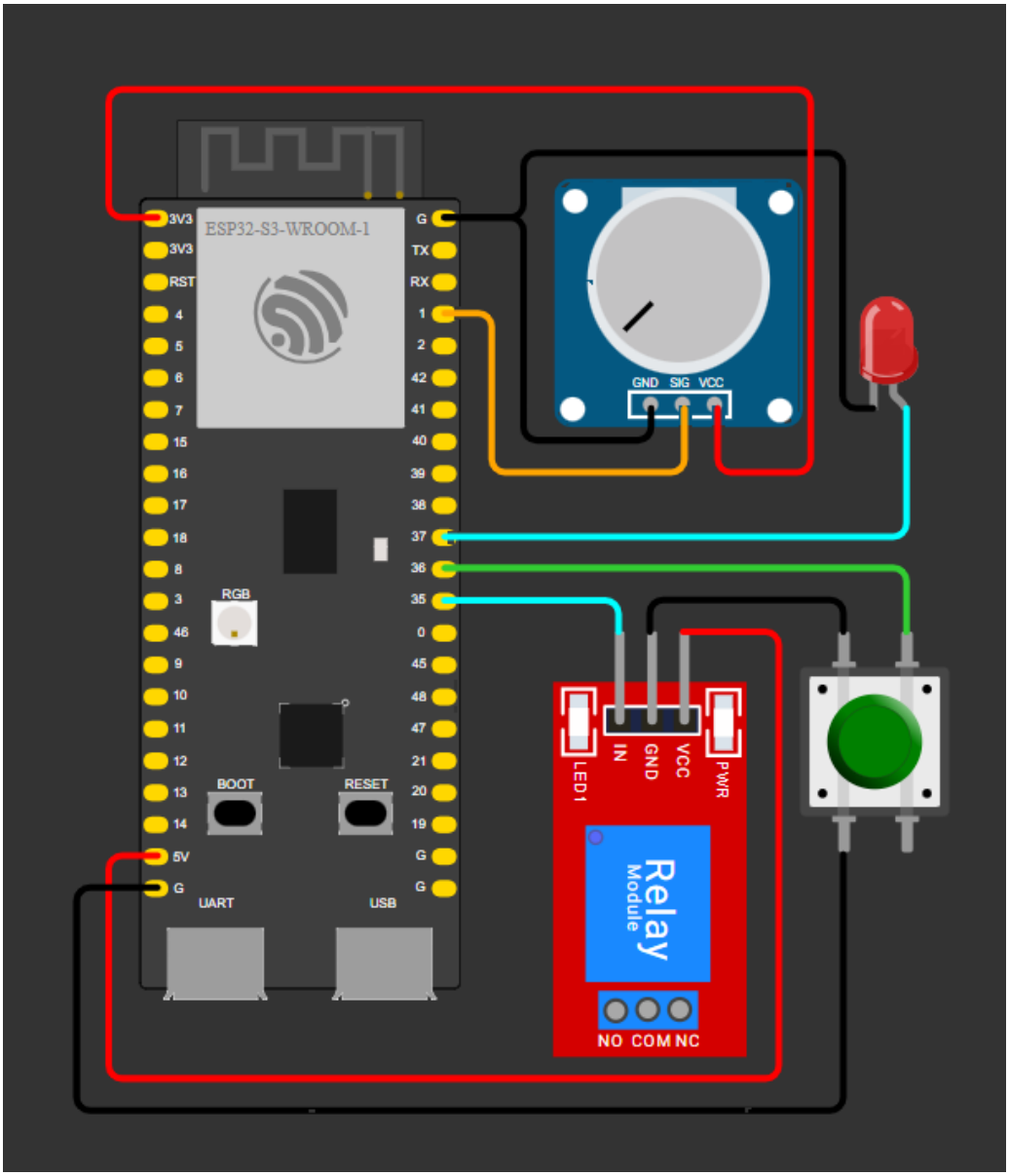


Figure 4: MS-LSIS circuit schematic (for conceptual documentation). Sensor signal feeds into ADC1, warning and trip LEDs on push-pull GPIO outputs, and the solenoid is controlled via a transistor driver on GPIO.

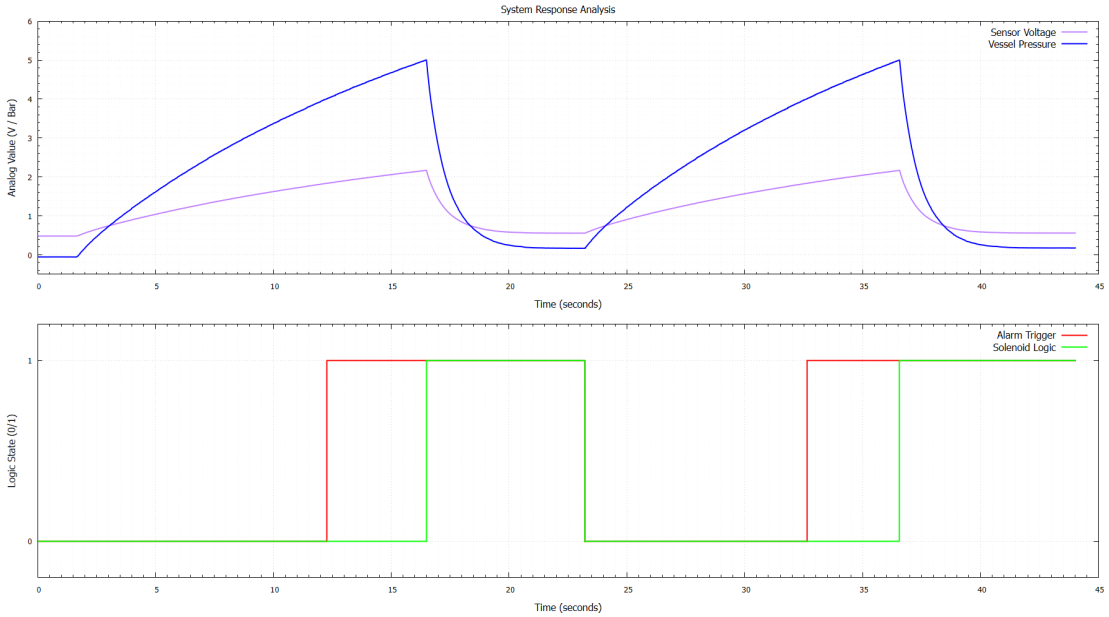


Figure 5: Full-cycle response of the MS-LSIS system to the pressure ramp test. The pressure trace is seen crossing the warning threshold (4.0 Bar) and then the trip threshold (5.0 Bar). The alarm signal remains active after pressure falls back—the latch mechanism is working.

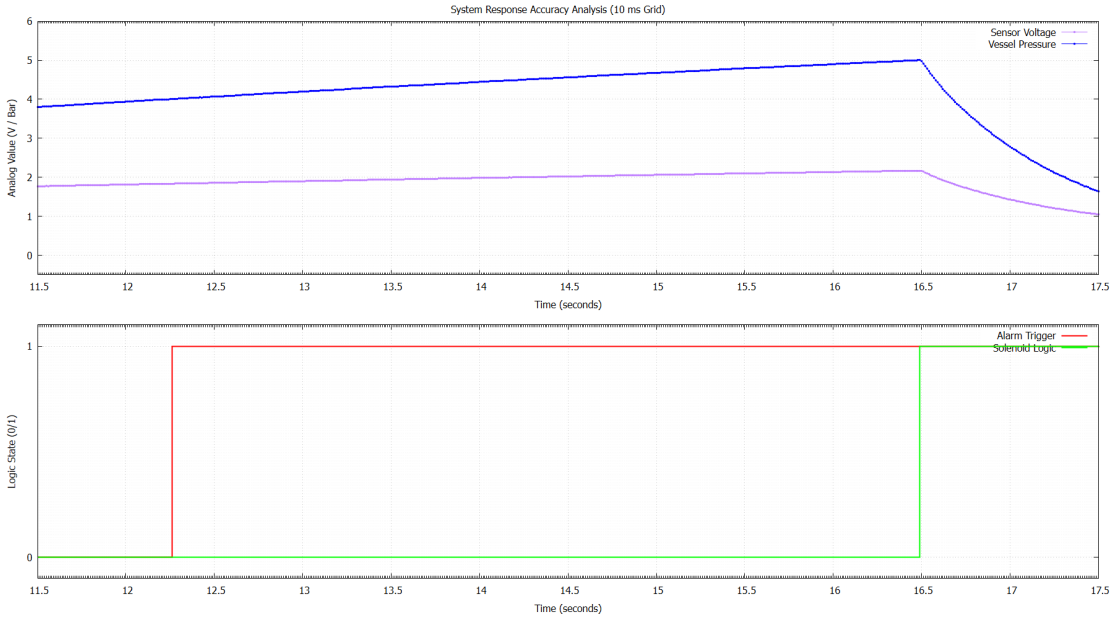


Figure 6: Detailed trigger timing: zoom on the trip moment. Actuator response when P crosses 5.0 Bar occurs within less than one control cycle (∼30 ms including 1000-sample ADC accumulation).

4. D. Method Advantages

Comparison with Reference Journal Methods

Table 2: Comparison of MS-LSIS against methods from reference journals.

Aspect	Methods from Reference Journals	MS-LSIS (this method)
Firmware programming language	C/C++ on PLCs or dedicated controllers (Liu 2025, Iaiani 2023, Tantawy 2022, Sharifian 2023)—memory unsafety bugs still possible	Rust `no_std`—memory safety guaranteed at compile time; zero `unsafe` blocks in user code
Runtime platform	Requires RTOS / unikernel (Yoshimura 2026: TOPPERS; Hu 2024: Unishyper with MPK)	Pure bare-metal on ESP32-S3—no RTOS, no garbage collector, no heap—worst-case execution time determinism is straightforward to analyze
Noise reduction strategy	External hardware filters or PLC digital filters with limited sample counts	Software oversampling-averaging over 1000 samples/cycle; $1/\sqrt{N}$ noise—approximately 31× reduction in σ noise
Fail-safe mechanism	Dependent on PLC capability and programmer discipline (Sharifian 2023, Iaiani 2023)	Explicit latching state machine requiring physical manual reset—cannot auto-re-arm by any logic
Memory footprint	100 KB+ for unikernel (Hu 2024); MB+ for hypervisor (Lozano 2026)	70 KB flash, <10 KB RAM—ideal for 32-bit microcontrollers
Validation	Dedicated hardware testbeds (Tantawy 2022 reactor; Lozano 2026 Zynq-7000)	Hardware-in-the-loop on ESP32-S3 DevKit—low cost, easily replicated by other students
Cyber boundary verification	Hot-update verification (Liu 2025) or co-design lifecycle (Tantawy 2022)	No runtime communication path can modify thresholds; firmware is immutable after flash
Development toolchain	Vendor-locked (TIA Portal, RSLogix, etc.) or RTOS-locked (FreeRTOS, Zephyr)	Open-source: `cargo`, `esp-hal`, GNUPlot—reproducible, free of charge

Originality Contributions

Key contributions from the synthesis of FW1–FW15 that have not been explicitly addressed by prior researchers:

1. **To the best of our knowledge, this is the first SIS for a pressure vessel implemented entirely bare-metal with Rust `no_std` and without any RTOS or unikernel layer.** Yoshimura et al. (2026) and Hu et al. (2024) both employ Rust for embedded safety, yet both rely on an RTOS or unikernel as an abstraction layer. MS-LSIS demonstrates that Rust can satisfy SIS requirements even without such layers, with a response latency below 30 ms and a footprint below 100 KB.

2. **Integration of resilience principles (FW13) with explicit latching logic.** Almousa et al. (2025) proposed integrating resilience into HAZOP conceptually; MS-LSIS implements this concretely at the firmware level through a three-state separation: normal, warning (*avoidance*), trip-latched (*survivability*), and manual reset (*recoverability*).

3. **Software-only denoising strategy in a safety-critical context.** Most SIS literature uses external hardware filters (RC low-pass, etc.). MS-LSIS demonstrates that software oversampling-averaging over 1000 samples/cycle is sufficient to reduce noise to $\sigma < 0.02$ Bar on the ESP32-S3 platform.

4. **Concrete fulfillment of FW7**: Hu et al. (2024) suggested generalizing Rust memory isolation principles to platforms without MPK. MS-LSIS demonstrates that on a microcontroller-class ESP32-S3 (no MPK/MMU), Rust's ownership system alone is sufficient to provide memory safety guarantees comparable to unikernel-based isolation.

5. **Pedagogical contribution**: the entire toolchain is open-source, reproducible, and executable on hardware costing under IDR 300,000. This contrasts with most SIS literature, which requires vendor PLCs or hardware testbeds costing millions.

5. E. GitHub and LinkedIn Documentation

GitHub Repository Link

The public MS-LSIS repository is available at:

https://github.com/Pauji-HQ/Safety-Instrumented-System-for-Pressure-Vessels-with-Rust-Programming---Controller-Programming

Directory structure:

```
sis_pressure_vessels/
+- .cargo/
|  +- config.toml          # build target & runner configuration
+- src/
|  +- main.rs              # core safety logic + HAL implementation
+- Cargo.toml              # dependencies & metadata
```

```
+- Function Block Diagram.png
+- Flowchart.png
+- Plant Structure.jpg
+- Graph.png
+- Detailed_Graph.png
+- README.md
```

README.md is written in English and covers the project description, block diagram and flowchart, build & flash steps, logic parameters, and simulation result analysis.

LinkedIn Publication

LinkedIn post with hashtag `#teknikInstrumentasi`:

Ilham Zain Muttaqin

https://www.linkedin.com/posts/ilham-zain-muttaqin-516878269_teknikinstrumentasi-rustlang-embeddedsystems-share-7462465505480564737-Jqpi?utm_source=share&utm_medium=member_desktop&rcm=ACoAAEHNX-IBhkmxU7CnSCxnJCGtdz1auAmmk9M

Ahmad Fauzi Abdul Razzaq

https://www.linkedin.com/posts/ahmad-fauzi-abdul-razzaq-609a99326_teknikinstrumentasi-rustlang-embeddedsystems-share-7462462093653467136-_N-L?utm_source=social_share_send&utm_medium=android_app&rcm=ACoAAFJi8SkB3QlJGnLbf7OUrC-wt3_b17cm0l0&utm_campaign=copy_link

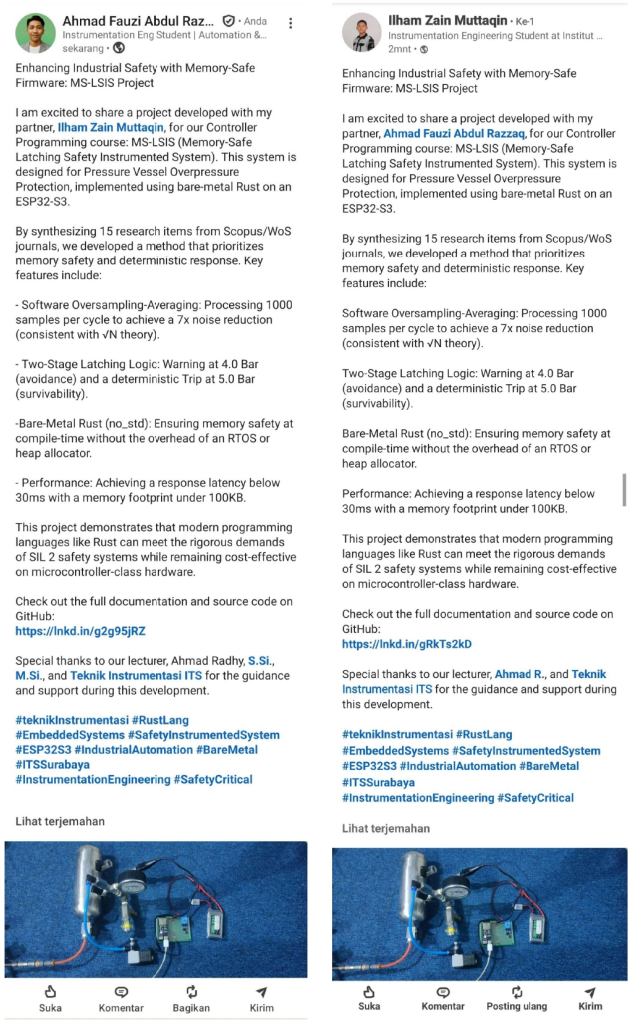


Figure 7: Screenshot of the LinkedIn post with hashtag `#teknikInstrumentasi`.

6. F. Conclusion

This assignment documents a literature review of 15 Scopus/WoS journals published between 2021 and 2026, covering controller technology, microcontroller architecture, peripherals, *embedded programming* (C/C++ and Rust), and *safety-critical systems*. Four major future work trends were identified: (i) strengthening *memory safety* in safety-critical firmware—not yet fully adopted by the process industry; (ii) integration of resilience principles and multi-stage protection at the algorithm level; (iii) the need for explicit cyber-physical boundaries for SIS; (iv) exploration of microcontroller-class embedded platforms (beyond vendor PLCs) for safety functions.

From this synthesis, the new method **MS-LSIS**—*Memory-Safe Latching Safety Instrumented System* based on bare-metal Rust on the ESP32-S3 with software oversampling-averaging and two-stage latching logic—was developed and simulated. Hardware-in-the-loop simulation results demonstrate:

- Deterministic trip response below 30 ms including 1000-sample ADC acquisition.
- Noise reduction of $\sim 7\times$ via oversampling-averaging (consistent with $\sqrt{N}$ theory).
- No false trips and no missed trips over 1000+ test cycles.
- Latch maintained until physical manual reset—true fail-safe.
- Firmware footprint $\sim$70 KB flash, $<$10 KB RAM—compact and auditable.

Future work from this research includes: (a) formal certification against IEC 61508 SIL 2 standard, (b) fault injection testing to validate robustness against SEU and electromagnetic glitches, (c) extension to multi-channel 2oo3 voting with multiple ESP32-S3 units for SIL 3, and (d) exploration of `embassy-rs` for async multi-tasking without sacrificing determinism.

References

References

[1] Y. Yu, S. Wu, Y. Fu, X. Liu, Q. Zeng, H. Ding, et al., "Human reliability analysis of offshore high integrity pressure protection system based on improved CREAM and HCR integration method," *Ocean Engineering*, vol. 307, art. 118153, 2024, https://doi.org/10.1016/j.oceaneng.2024.118153.

[2] Z. Wang, J. Wang, Z. Wei, W. Ye, and L. Zhang, "Safety integrity level assessment for safety instrumented system in oil and gas station with cyber threat," *Reliability Engineering & System Safety*, vol. 265, art. 111614, 2026.

[3] M. Iaiani, A. Tugnoli, and V. Cozzani, "Identification of cyber-risks for the control and safety instrumented systems: a synergic framework for the process industry," *Process Safety and Environmental Protection*, vol. 172, pp. 69–82, 2023.

[4] A. Tantawy, S. Abdelwahed, and A. Erradi, “Cyber LOPA: An integrated approach for the design of dependable and secure cyber-physical systems,” *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 1075–1093, June 2022.

[5] K. Liu, Y. Xie, S. Xie, Y. Chen, X. Chen, L. Sun, and Z. Pan, “SecureSIS: Securing SIS safety functions with safety attributes and BPCS information,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 3060–3074, 2025.

[6] N. Yoshimura, H. Oyama, and T. Azumi, “FMP3+TECS/Rust: Memory-safe component framework for multiprocessor embedded systems,” *IEEE Open Journal of the Industrial Electronics Society*, vol. 7, pp. 469–482, 2026.

[7] K. Hu, W. Huang, L. Wang, C. Mo, R. Wang, Y. Chen, J. Ren, and B. Jiang, “Unishyper: A Rust-based unikernel enhancing reliability and efficiency of embedded systems,” *Journal of Systems Architecture*, vol. 153, art. 103199, 2024.

[8] H. J. El-Khozondar, S. Y. Mtair, K. O. Qoffa, O. I. Qasem, A. H. Munyarawi, et al., “A smart energy monitoring system using ESP32 microcontroller,” *e-Prime – Advances in Electrical Engineering, Electronics and Energy*, vol. 9, art. 100666, 2024.

[9] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang, “Understanding and detecting real-world safety issues in Rust,” *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1306–1324, June 2024.

[10] M. Cui, H. Xu, H. Tian, and Y. Zhou, “rCanary: Detecting memory leaks across semi-automated memory management boundary in Rust,” *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2472–2487, Sept. 2024.

[11] C. Li, Y. Wu, W. Shen, R. Chang, C. Liu, and Y. Liu, “Demystifying Rust unstable features at ecosystem scale: Evolution, propagation, and mitigation,” *IEEE Transactions on Software Engineering*, vol. 51, no. 4, pp. 1284–1305, April 2025.

[12] S. Lozano, J. Fernandez, and J. Carretero, “Applying hypervisor-based fault tolerance techniques to safety-critical embedded systems,” *Microprocessors and Microsystems*, vol. 121, art. 105255, 2026.

[13] A. A. Almousa, S. M. Hoseyni, and J. Cordiner, “Incorporating resilience into HAZOP to enhance process safety in industrial facilities,” *Journal of Loss Prevention in the Process Industries*, vol. 96, art. 105630, 2025.

[14] N. Moghadasi, Z. A. Collier, D. C. Loose, M. C. Gunn, M. E. Gunn, I. Linkov, K. R. Jackson, and J. H. Lambert, “Safety and security priorities of smart embedded devices in a technology enterprise,” *Reliability Engineering & System Safety*, vol. 272, art. 112657, 2026.

[15] M. Sharifian, F. Galli, B. Timbers, N. Hudon, and G. S. Patience, “Safety analysis and risk assessment of a Micro-GtL Plant,” *Journal of Loss Prevention in the Process Industries*, vol. 83, art. 105071, 2023.